

FASTAPI

Autenticação com OAuth2PasswordBearer

Da instalação à primeira API

```
pip install fastapi
fastapi dev main.py
fast
```

Criar aplicação API RESTFUL com
fastapi com GET e POST

Importando OAuth2PasswordBearer

```
from fastapi import
    FastAPI, HTTPException, Depends
from fastapi.security import
    OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme =
OAuth2PasswordBearer(tokenUrl="token")
...
```



oauth2_scheme =
OAuth2PasswordBearer(tokenUrl="token")

- Esquema de autenticação OAuth2 utilizando **Bearer Token**
- Quando utilizarmos esse objeto como uma dependência, FastAPI poderá procurar o token enviado no cabeçalho:
 - **Authorization: Bearer token-fake-admin**
- O parâmetro tokenUrl="token" indica qual endpoint será utilizado pelo cliente para obter o token
- OAuth2PasswordBearer(tokenUrl="token") não cria automaticamente a rota /token.
 - O tokenUrl informa ao esquema OAuth2 e à documentação da API onde o cliente poderá solicitar o token

Criando usuário fake

```
from fastapi import
    FastAPI, HTTPException, Depends
from fastapi.security import
    OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme =
    OAuth2PasswordBearer(tokenUrl="token")

USUARIO_FAKE = {
    "username": "admin",
    "password": "1234"
}

TOKEN_FAKE = "token-fake-admin"
```



Segurança

- Armazenar senha dessa forma é propositalmente inseguro e está sendo feito apenas para que possamos enxergar o fluxo da autenticação sem introduzir hashing ou JWT

Criando a rota de login

```
from fastapi import FastAPI, HTTPException, Request, Depends
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel
from produto import Produto

app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")

USUARIO_FAKE = {"username": "admin", "password": "1234"}
TOKEN_FAKE = "token-fake-admin"

@app.post("/token")
def login(form_data: OAuth2PasswordRequestForm =
Depends(OAuth2PasswordRequestForm)):
    if form_data.username != USUARIO_FAKE["username"] or \
        form_data.password != USUARIO_FAKE["password"]:
        raise HTTPException(status_code=401, detail="Usuário ou
senha inválidos.")
    return {
        "access_token": TOKEN_FAKE,
        "token_type": "Bearer"
    }

...
// Demais rotas
```

form_data: OAuth2PasswordRequestForm
=

Depends(OAuth2PasswordRequestForm)

- FastAPI recebe os dados enviados pelo cliente e disponibiliza:
 - form_data.username
 - form_data.password

var: class = Depends(method)

- Antes de executar a função login, execute a lógica representada por method e coloque o resultado na variável var.

Testando o token

```
@app.get("/teste-token")
def teste_token(token: str = Depends(oauth2_scheme)):
    return {"token_recebido": token}
```



Rota teste-token

- Cliente envia:
 - Authorization: Bearer t
- FastAPI utiliza oauth2_scheme para obter o conteúdo do token
- teste_token recebe
 - token = "token-fake-admin"
- teste-token envia:
 - {
 "token_recebido": "token-fake-admin"
}



Botão Authorize no Swagger

- Swagger cria botão "Authorize" para entrar com username e password

Função de validação de token

```
def validar_token(token: str = Depends(oauth2_scheme)):\n\n    if token != TOKEN_FAKE:\n        raise HTTPException(\n            status_code=401,\n            detail="Token inválido",\n            headers={"WWW-Authenticate": "Bearer"}\n        )\n\n    return token
```



validar_token

- Função para comparar o token recebido pelo usuário com o token real

Protegendo uma rota

```
@app.get("/listar_produtos_autenticado/")
def listar_produtos(limite: int = 10, token: str =
Depends(validar_token)):
    return produtos[:limite]
```



token: str = Depends(validar_token)

- Executa a função `listar_produtos_autenticado` somente após `validar_token`

Segurança com JWT

Da instalação à primeira API

```
pip install fastapi  
fastapi dev main.py  
fast
```

Criar aplicação API RESTFUL com
fastapi com GET e POST

JWT (JSON Web Token)



JWT (JSON Web Token)

- representação compacta de informações, chamadas claims, que pode ser assinada para permitir a verificação de sua integridade



Biblioteca PyJWT

- permite codificar e decodificar JSON Web Tokens em Python



Dinâmico

- O valor real será maior e poderá mudar a cada login dependendo dos dados utilizados para criá-lo.



Verificação

- Ao invés:
 - token-fake-admin
- Tem-se:
 - eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbG1mV4cCI6MTc... .abc123...



instalação

- `py -m pip install pyjwt`

Importando PyJWT

```
from datetime import datetime, timedelta, timezone
import jwt
from jwt.exceptions import InvalidTokenError
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm

from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel
```



Imports

- `datetime`: usado para registrar o momento de criação ou o limite de validade do token.
- `timedelta`: usado para somar minutos ou dias à data atual (ex: definir que o token dura 30 minutos).
- `timezone`: Gerencia fusos horários, garantindo que o cálculo de expiração do token seja preciso independentemente de onde o servidor esteja hospedado (usando o padrão UTC).
- `InvalidTokenError`: É o aviso de erro disparado automaticamente pelo jwt caso o sistema tente ler um token que foi falsificado ou adulterado.

Criando uma função gerar_token()

```
def gerar_token(username: str):  
    expiracao = datetime.now(timezone.utc) +  
    timedelta(minutes=30)  
  
    dados = {  
        "sub": username,  
        "exp": expiracao  
    }  
  
    token = jwt.encode(  
        dados,  
        SECRET_KEY,  
        algorithm=HS256  
    )  
  
    return token
```



função gerar_token

- A função recebe username
- expiracao é o momento de expiração do token
 - expiração = agora + 30 minutos
- sub (subject): identifica quem é o dono do token (geralmente o username ou ID do usuário)
- exp (expiration): identifica quando o token expira



função jwt.encode(dados, chave, algoritmo)

- dados: o conteúdo do token (payload), tipo sub e exp. Não é secreto, só codificado.
- SECRET_KEY: a chave usada pra assinar o token, garantindo que ninguém adultere o conteúdo sem ser detectado. Fica só no servidor.
- algorithm: qual algoritmo criptográfico gera essa assinatura (ex: "HS256").

Função validar_token_jwt()

```
def validar_token_jwt(token: str = Depends(oauth2_scheme)):  
  
    erro = HTTPException(  
        status_code=401,  
        detail="Token inválido ou expirado",  
        headers={"WWW-Authenticate": "Bearer"}  
    )  
  
    try:  
  
        payload = jwt.decode(  
            token,  
            SECRET_KEY,  
            algorithms=["HS256"]  
        )  
  
        username = payload.get("sub")  
  
        if username is None:  
            raise erro  
  
    except InvalidTokenError:  
        raise erro  
  
    return username
```

Nova rota com JWT

```
@app.post("/token_jwt")
def login(form_data: OAuth2PasswordRequestForm =
Depends(OAuth2PasswordRequestForm )):

    if (
        form_data.username != USUARIO_FAKE["username"]
        or form_data.password !=
USUARIO_FAKE["password"]
    ):
        raise HTTPException(
            status_code=401,
            detail="Usuário ou senha inválidos"
        )

    token = gerar_token(form_data.username)

    return {
        "access_token": token,
        "token_type": "bearer"
    }
```

DFD Básico



DFD - (Data Flow Diagram)

- DFD — Data Flow Diagram, ou Diagrama de Fluxo de Dados, representa:
 - quem envia dados;
 - quem recebe dados;
 - por onde os dados passam;
 - onde os dados são processados;
 - onde os dados são armazenados.
- Em segurança, o DFD ajuda a identificar onde um atacante pode tentar acessar, modificar ou interromper informações.



DFD - (Data Flow Diagram)

- Componentes DFD:
 - Entidade externa: quem está fora do sistema e interage com ele
 - Exemplo: Cliente
 - Processo: algo que recebe e processa dados
 - Exemplo: FastAPI
 - Data Store: onde os dados ficam armazenados
 - Exemplo: Banco de Dados, SQLite
 - Fluxo de dados: informação que passa de um componente para outro
 - Exemplo: POST /auth/token, username + password
- Entradas: Dados que entram no sistema
 - Exemplos: username, password, token JWT, texto para previsão, ID da prediction
- Saídas: Dados que saem do sistema
 - Exemplos: { "access_token": "eyJ...", { "id": 15, "category", "delivery_problem"}



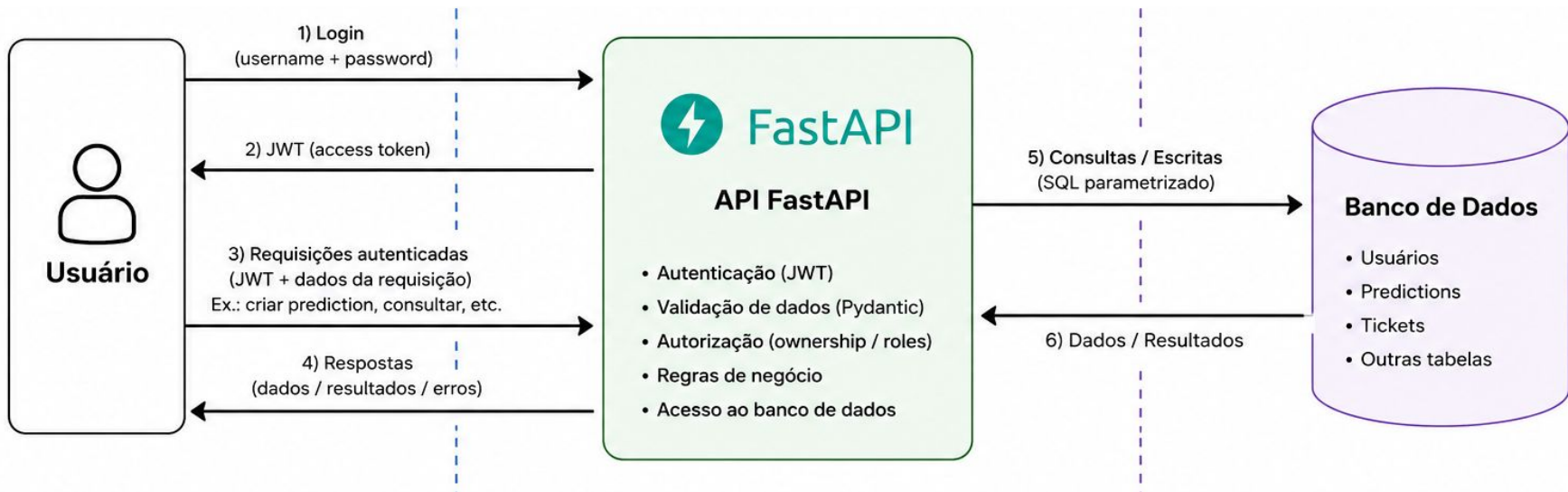
DFD - (Data Flow Diagram)

- Entradas: Dados que entram no sistema
 - Exemplos: username, password, token JWT, texto para previsão, ID da prediction
- Saídas: Dados que saem do sistema
 - Exemplos: { "access_token": "eyJ..." }, { "id": 15, "category", "delivery_problem" }
- Trust Boundary, ou fronteira de confiança, é um ponto onde os dados passam entre ambientes com níveis diferentes de confiança
 - Exemplo: dados enviados do usuário para API



Representação DFD

- Representação DFD: utilizar ferramenta **draw.io**
 - Processo/Usuário: retângulo
 - Banco de dados: cilindro
 - Fluxo dos dados: setas
 - Trust Boundary: linha tracejada cortando um fluxo de dados



Tríade CIA

- Cada componente do DFD deve ser classificado de acordo com a tríade CIA:
 - C (Confidentiality) - confidencialidade: Quem pode ver essa informação?
 - I (Integrity) - integridade: Quem pode modificar essa informação?
 - A (Availability) - disponibilidade: O serviço precisa continuar funcionando?
- Exemplo login:
 - Usuário:
 - Confidencialidade: Credenciais e JWT não devem ser expostos
 - Integridade: Requisições precisam ser validadas
 - Disponibilidade: API deve permanecer acessível

